

Cluster: AIML



TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS

Natural Language to OKS Query Translation Module

SUBMITTED BY

Asmit Khanal (PUL08BCT017)
Chandan Kumar Shah (PUL080BCT023)
Aditya Kumar Shah (PUL080BCT011)
Rhythm Adhikari (PUL080BCT065)

A

BE MINOR PROJECT FINAL REPORT

SUBMITTED TO THE

DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING
IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE DEGREE OF
BACHELOR OF ENGINEERING IN
COMPUTER ENGINEERING

DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING
LALITPUR, NEPAL

AUGUST 26, 2026

DECLARATION OF AUTHORSHIP

This is to certify that the minor project entitled “**Natural Language to OKS Query Translation Module**” submitted for the degree of Bachelor of Engineering in Computer Engineering comprises our original work and no part of this work has been used for the award of another course or degree. We declare that this work is our own. If any part of the work is not our own, we have indicated it by acknowledging the source of that part or those part of the work with proper citations.

Name of the Student

Signature

Asmit Khanal

Chandan Kumar Shah

Aditya Kumar Shah

Rhythm Adhikari

Date: August 26, 2026

APPROVAL

The undersigned certify that they have read, and recommended to the Institute of Engineering for acceptance, a minor project entitled “**Natural Language to OKS Query Translation Module**” submitted by Asmit Khanal (PUL08BCT017), Chandan Kumar Shah (PUL080BCT023), Aditya Kumar Shah (PUL080BCT011), and Rhythm Adhikari (PUL080BCT065) in partial fulfillment of the requirements for the degree of Bachelor of Engineering in Computer Engineering.

Supervisor, Name of the Supervisor

Designation,

Department of Electronics & Computer Engineering

Pulchowk Campus, IOE, TU

Internal Examiner,

Department of Electronics & Computer Engineering

Pulchowk Campus, IOE, TU

Head of Department, Prof. Dr. Ram Krishna Maharjan

Department of Electronics & Computer Engineering,

Pulchowk Campus, IOE, TU

Date: August 26, 2026

COPYRIGHT

The authors have agreed that the library, Department of Electronics & Computer Engineering, Pulchowk Campus, Institute of Engineering may make this project report freely available for inspection. Moreover, the authors have agreed that permission for extensive copying of this report for scholarly purpose may be granted by the professor(s) who supervised the work recorded herein or, in their absence, by the Head of the Department wherein the project was done. It is understood that the recognition will be given to the author of this report and to the Department of Electronics & Computer Engineering, Pulchowk Campus, and Institute of Engineering in any use of the material of this project. Copying or publication or the other use of this report for financial gain without approval of the Department of Electronics & Computer Engineering, Pulchowk Campus, Institute of Engineering and authors' written permission is prohibited.

Request for permission to copy or to make any other use of the material in this report in whole or in part should be addressed to:

Head of Department
Department of Electronics & Computer Engineering
Pulchowk Campus
Pulchowk, Lalitpur
Nepal

ABSTRACT

[Content to be added...]

ACKNOWLEDGEMENTS

[Content to be added...]

TABLE OF CONTENTS

DECLARATION	i
APPROVAL	ii
COPYRIGHT	iii
ABSTRACT	iv
ACKNOWLEDGEMENTS	v
LIST OF FIGURES	ix
LIST OF TABLES	ix
ABBREVIATIONS	x
1 INTRODUCTION	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Objectives	3
1.3.1 Main Objective	3
1.3.2 Specific Objectives	3
1.4 Rationale	3
1.5 Scope	4
1.6 Limitations	4
1.7 Outline of the Report	4
2 LITERATURE REVIEW	5
2.1 Review of Related Theory	5
2.2 Review of Related Works	5
2.3 Gap analysis(if required)	5
3 METHODOLOGY	6
3.1 Project approach	6

3.2	System Architecture	7
3.3	System Design/Modules (UI module, backend logic, database, APIs, etc)	9
3.3.1	Use-Case Diagram	10
3.3.2	Context Diagram (DFD Level 0)	12
3.3.3	Class Diagram	12
3.3.4	Sequence Diagram	13
3.4	Technology Stack (languages, frameworks, tools, etc)	14
3.5	Development Methodology (agile/waterfall/iterative or suitable one) . .	14
3.6	Data Handling (input, storage, processing)	15
3.6.1	Input and Storage	15
3.6.2	Processing and Context Management	15
3.6.3	Level-1 Data Flow	15
3.6.4	Level-2 Translation and Validation Flow	15
3.7	Testing and Validation (unit testing, integration testing, user testing) . .	16
3.8	Deployment Setup (hosting, server, environment)	17
4	IMPLEMENTATION DETAILS	18
4.1	Requirements Specification(functional/non-functional requirements, constraints)	18
4.2	System Overview (Description of implemented system, mapping methodology to implementation)	18
4.3	Development Environment(programming languages, tools, libraries and frameworks used, hardware and software specifications)	18
4.4	System Implementation(data/input handling, pre-processing steps, algo- rithm implementation, modules design and integration etc)	18
4.5	Interface and Visualization	18
4.6	Deployment and Execution(system setup, execution steps, lo- cal/server/cloud setup)	18
4.7	Challenges and Solutions(technical challenges faced, solutions found) .	18
5	RESULTS AND DISCUSSION	19
5.1	Results	19
5.2	Discussion	19

5.3	Comparative analysis(if applicable)	19
6	CONCLUSIONS AND RECOMMENDATIONS	20
6.1	Conclusions	20
6.2	Recommendations/Future Enhancements	20
	REFERENCES	20
	APPENDICES	22

LIST OF FIGURES

3.1	Deployment and processing architecture of the OKS MCP server.	8
3.2	Use-case diagram of the OKS MCP Server.	11
3.3	Level-0 context data-flow diagram of the OKS MCP Server.	12
3.4	Principal classes and collaborations in the merged implementation. . . .	13
3.5	Concise sequence of operations for an MCP query request.	13
3.6	Level-1 data-flow diagram for OKS MCP request processing.	15
3.7	Level-2 data-flow diagram for translation and validation.	16

LIST OF TABLES

ABBREVIATIONS

API Application Programming Interface

LLM Large Language Model

OKS Object Kernel Support

CHAPTER 1

INTRODUCTION

1.1 Background

The ATLAS experiment at CERN’s Large Hadron Collider (LHC) relies on a highly complex data acquisition (DAQ) system to manage thousands of hardware and software components in real time. The configuration of this system, spanning detector electronics, readout chains, and computing farms, is managed by the Object Kernel Support (OKS) framework, a persistent in-memory object database built for the ATLAS Trigger and Data Acquisition (TDAQ) system [3]. OKS provides a schema-driven, object-oriented model that represents complex topologies as interconnected object graphs [3].

To meet the operational demands of successive LHC runs, ATLAS configuration management evolved from monolithic databases [2] to a Git-based service for LHC Run 3 [1]. This modern architecture provides distributed version control and full provenance tracking, ensuring every configuration state is a Git commit and the entire detector history is queryable [1].

Despite this mature infrastructure, querying the OKS database remains a specialist task. The proprietary OksQuery language uses a strict, S-expression-like syntax (e.g., (all ("InitTimeout" "2" >))) that demands detailed knowledge of class hierarchies, relationship paths, and operator semantics. Consequently, manual query formulation by shifters is error-prone, time-consuming, and inaccessible to non-specialists.

The emergence of Large Language Models (LLMs) and the Model Context Protocol (MCP) offers a new paradigm for AI-driven operations, enabling agents to interact with external databases through a standardized interface. An MCP server for the ATLAS DAQ is currently under development to allow LLMs to search the OKS database. However, a critical challenge arises: the full configuration, comprising hundreds of schema classes and thousands of objects, far exceeds any LLM’s context window. Thus, the MCP server must filter data on the backend, requiring the LLM to generate syntactically perfect OksQuery strings without seeing the complete schema.

This project addresses this challenge by developing a Natural Language to OksQuery translation module. Architected as a Retrieval-Augmented Generation (RAG) system, it

utilizes a “schema-RAG” approach where the retrieval corpus is the live, version-scoped OKS schema itself. This ensures the LLM receives only the relevant schema slice (1–3 classes) rather than the full schema, which can exceed 500 classes depending on the TDAQ release and partition, satisfying both accuracy and context-window constraints. The system is deployed securely within the CERN environment via SSH-based hosting with port tunneling, exposing the MCP server to external LLM agents without requiring them to reside inside the CERN network perimeter.

1.2 Problem Statement

The Object Kernel Support (OKS) framework is a persistent in-memory object database that provides a schema-driven, object-oriented configuration model. By supporting classes, attributes, inheritance, and composite objects, it enables the precise representation of complex hardware and software topologies. However, interacting with OKS remains a highly specialized task. Users must formulate queries using the proprietary OksQuery language or low-level C++ and Python APIs, which demands detailed, manual knowledge of class hierarchies, object states, relationship paths, and strict operator semantics.

While general-purpose Large Language Models (LLMs) present an opportunity to automate this process, they are fundamentally incapable of generating accurate OksQuery statements out-of-the-box. This failure is driven by a complete lack of parametric context regarding the specific classes, attributes, and objects within the OKS schema. Furthermore, the configuration is strictly version-specific, with schema definitions evolving across different software releases and Git commits. Because LLMs lack access to this dynamic, version-scoped context, they frequently hallucinate non-existent classes or invalid relationships, rendering their raw query accuracy insufficient for operational deployment.

To address this, we built a query translation module hosted as an MCP server via SSH port tunneling, using a version-scoped schema-RAG pipeline that injects only the relevant OKS schema slice into the LLM prompt. A deterministic validate/repair loop then checks every generated OksQuery against the authoritative schema before execution, ensuring reliable accuracy without the full configuration entering the context window.

1.3 Objectives

1.3.1 Main Objective

The main objective of this project is to develop an MCP server capable of translating natural-language questions into validated OksQuery statements through a query translation module with an integrated validate/repair loop, deployable as a tool for external LLM agents.

1.3.2 Specific Objectives

The specific objectives of this project are:

- (i) To develop a version-scoped pipeline that translates natural language into OksQuery strings by retrieving targeted OKS schema slices and enforcing syntactic correctness through a deterministic validate/repair loop.
- (ii) To deploy the translation module as a Model Context Protocol (MCP) server within the CERN compute infrastructure, utilizing SSH port tunneling to securely expose the live TDAQ configuration to external LLM agents.
- (iii) To evaluate translation accuracy by executing generated queries against the native C++ backend and analyzing performance metrics such as valid-syntax rate, result-set equivalence, and execution latency across stratified query difficulties.

1.4 Rationale

The ATLAS Data Acquisition (DAQ) system relies on the Object Kernel Support (OKS) framework, a complex, version-controlled object database. Querying this configuration during time-critical operations requires specialized knowledge of the proprietary OksQuery language, creating a significant bottleneck for control room operators.

While Large Language Models (LLMs) could automate this querying process through natural language, raw models fail because they hallucinate syntax for undocumented, proprietary languages and cannot ingest massive, constantly evolving schemas within their context limits.

This project addresses these fundamental limitations by developing a retrieval-augmented translation module deployed as a Model Context Protocol (MCP) server. By utilizing a version-scoped Schema-RAG pipeline, the system feeds the LLM only the

exact schema slice it needs. Furthermore, by forcing the LLM to output an Abstract Syntax Tree (AST) that undergoes a strict validate-and-repair loop against the authoritative, version-bound OKS schema, the system eliminates syntax hallucinations before execution. Ultimately, this establishes a robust and verifiable paradigm for safely deploying generative AI within high-stakes scientific infrastructure.

1.5 Scope

The system is scoped to translating natural-language questions about the ATLAS DAQ configuration into validated OksQuery statements for read-only retrieval, processing one query at a time within a single, explicitly resolved configuration version (TDAQ release, Git revision, or run number), with schema retrieval performed dynamically per request using a version-scoped lexical pipeline (exact match, alias lookup, BM25) keyed by schema fingerprint, and the retrieval index regenerated from the OKS schema whenever the configuration changes. The module generates an intermediate AST representation rather than raw OksQuery strings, validated deterministically against the version-bound OKS schema before compilation, and is deployed as an MCP server hosted within CERN compute infrastructure and exposed to external LLM agents through SSH port tunneling, with conversational persistence and session state managed by the consuming LLM agent rather than the translation module. The system does not write, modify, or delete configuration data, does not provide operational recommendations or interpret query results beyond factual reporting, does not access live running partition memory or process telemetry data, does not handle user authentication (security relies on the CERN SSH tunnel and institutional network perimeter), and does not support non-English natural-language input or cross-version comparison queries.

1.6 Limitations

[Content to be added...]

1.7 Outline of the Report

[Content to be added...]

CHAPTER 2

LITERATURE REVIEW

2.1 Review of Related Theory

[Content to be added...]

2.2 Review of Related Works

[Content to be added...]

2.3 Gap analysis(if required)

[Content to be added...]

CHAPTER 3

METHODOLOGY

3.1 Project approach

The project was developed through successive capability increments and repeated verification against the OKS runtime. The work began with a translation prototype and was refined when testing exposed limitations in query generation, schema awareness, version handling, and execution. Each refinement preserved the working components while adding a more reliable capability. The final implementation therefore combines an iterative development process with an incremental growth of functionality, rather than treating the system as a single implementation completed at the end of the project.

The principal stages of the approach were as follows:

- I. **Domain and runtime analysis.** The OKS object model, schema files, OksQuery grammar, Python config binding, and native oks_dump utility were examined first. This established the supported classes, attributes, relationships, operators, query constraints, and runtime requirements.
- II. **Baseline translation prototype.** A basic natural-language to OksQuery path was implemented to test the feasibility of using an OpenAI-compatible language model for query generation. Initial outputs were inspected and executed against the native runtime so that syntactic and semantic failure cases could be identified.
- III. **Version-scoped context construction.** The approach was then extended to resolve current and historical configuration versions. For each request, the context builder creates an immutable OksContext with version metadata, schema and data locations, and a schema fingerprint. The retrieval index is consequently aligned with the configuration used for execution.
- IV. **Structured translation and validation.** Instead of asking the language model to produce executable query text directly, the system requests a JSON intermediate representation. The representation is normalised, checked structurally and semantically against the active schema, repaired within a bounded retry loop when required, and compiled deterministically to OksQuery.
- V. **Execution, service integration, and verification.** Validated queries are ex-

ecuted through the Python `config` interface or the `oks_dump` fallback. The pipeline was subsequently exposed through the stateless MCP service, which provides query, translation-only, and environment-probe tools. Unit, integration, and MCP contract tests were used to verify the individual modules and their combined behaviour.

3.2 System Architecture

The system follows a service-oriented pipeline deployed across the user environment and the CERN TDAQ environment. A user submits a natural-language question through a coding agent or another MCP-compatible client. The client connects to a local MCP endpoint, which is securely forwarded to the MCP server running on CERN lxplus through an SSH local port forward. This keeps the remote service private and avoids exposing a public application port.

Within the CERN environment, the Streamable HTTP MCP server exposes the `oks_query`, `oks_translate`, and `oks_environment_probe` tools. `OksQueryService` validates inputs, applies request and result limits, serialises the request, and returns structured responses. The `OksPipeline` resolves the requested configuration, retrieves the relevant version-scoped schema context, and coordinates translation and execution.

The translation and validation stage communicates with an OpenAI-compatible LLM API over HTTPS. The generated representation is checked and repaired when necessary before compilation. The executor then accesses the approved OKS data through the TDAQ Python `config` binding or the native `oks_dump` fallback. Only filtered structured results and diagnostic metadata are returned to the MCP client; conversational memory remains with the calling agent.

The deployment and request flow are summarised in Figure 3.1.

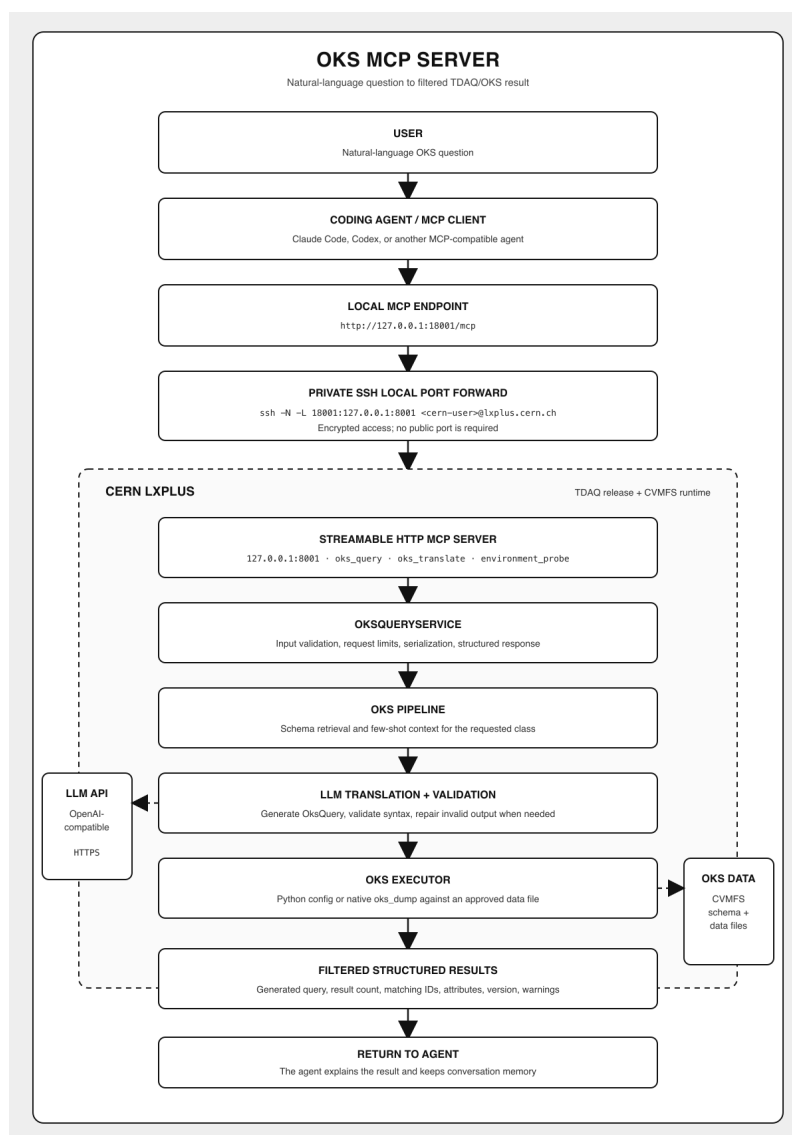


Figure 3.1: Deployment and processing architecture of the OKS MCP server.

3.3 System Design/Modules (UI module, backend logic, database, APIs, etc)

The system is a layered, read-only translation and execution pipeline. It separates request handling, version resolution, schema retrieval, model interaction, deterministic validation, execution, and MCP service access so that each responsibility can be tested independently.

- I. **Request and intent module.** CLI, programmatic, and MCP entry points validate the question, version selector, and service limits. Intent classification separates query, help, and out-of-scope requests.
- II. **Version and context module.** The resolver interprets current or historical tags, hashes, dates, and run identifiers. The context builder creates the version-bound `OksContext`, with a request lock protecting historical runtime selection.
- III. **Schema retrieval module.** The provider reads classes, attributes, inheritance, and relationships from the active schema. Exact and lexical matching, synonym hints, and relationship expansion build a compact, version-fingerprinted schema context.
- IV. **Translation and validation module.** The prompt builder combines the question, schema context, rules, and examples. The translator creates a structured representation; `oks_ast` validates it and the compiler deterministically produces `OksQuery` text. Invalid output enters a bounded repair loop.
- V. **Execution and interpretation module.** The executor runs only validated queries through the TDAQ Python `config` binding or `oks_dump`. Results include query, version, target class, count, status, and warnings. CLI output may be interpreted in the application, while MCP returns structured data to the calling agent.
- VI. **MCP service module.** `OksQueryService` provides a stateless boundary with input and version checks, result limits, serialisation, and controlled errors. `FastMCP` exposes `oks_query`, `oks_translate`, and `oks_environment_probe`; configuration remains in OKS/TDAQ.

3.3.1 Use-Case Diagram

The use-case view identifies the developer or team member and coding agent as the main actors. The LLM API supports translation and repair, while the server handles MCP access, versioned queries, validation, environment probing, and read-only execution. Conversation memory and follow-up resolution remain with the host agent; the server exposes only approved data and filtered results.

Use-Case Descriptions

UC-01: Query OKS Data **Actor:** User, programmatic client, or MCP client.

Precondition: The service is configured and the question and selected configuration are valid.

Postcondition: Structured results, status, and warnings are returned; the OKS configuration is unchanged.

Flow: Validate the request, resolve context, retrieve schema, generate and validate the representation, compile OksQuery, execute it, and return the normalised result. Unresolved versions, runtime failures, or exhausted repair attempts stop execution with a controlled error.

UC-02: Translate Without Execution **Actor:** User, developer, or MCP client.

Precondition: The question, optional selector, schema context, and LLM endpoint are available.

Postcondition: A validated representation and compiled OksQuery are returned without contacting the OKS runtime.

Flow: Resolve context, retrieve schema, generate, validate, and if necessary repair the representation within the retry limit. Invalid input, missing schema data, or failed repair returns a controlled error.

UC-03: Probe OKS Environment **Actor:** User, operator, or MCP client.

Precondition: Runtime paths and settings are loaded.

Postcondition: A non-secret readiness report covers dependencies, paths, schema availability, and classes; no query or mutation occurs.

Flow: Check the Python config path and/or oks_dump; report missing dependencies

as diagnostics.

Included use cases **UC-04: Resolve Context** interprets a current or historical selector and creates a version-bound `OksContext`. **UC-05: Validate and Compile** normalises the JSON representation, checks it against the active schema, and compiles it only after validation; invalid output is repaired within a bound. **UC-06: Execute Query** runs the validated expression through the Python config binding or `oks_dump` and returns a read-only `ExecutionResult`.

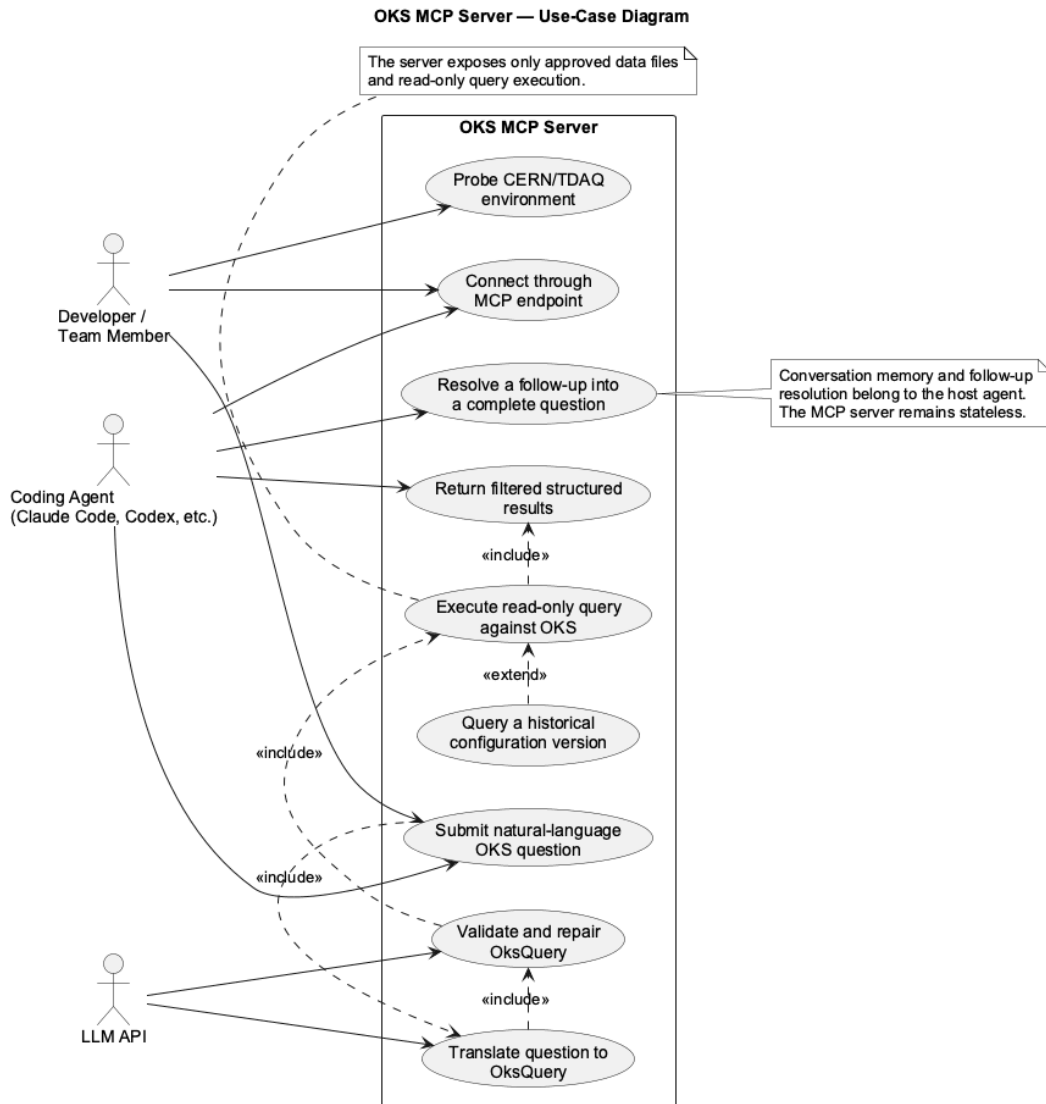


Figure 3.2: Use-case diagram of the OKS MCP Server.

3.3.2 Context Diagram (DFD Level 0)

The Level-0 DFD models the OKS MCP Server as one process interacting with the developer or coding agent, the LLM API, and the CERN TDAQ/OKS configuration. It receives a question and version, retrieves schema context, validates the candidate query, executes only an approved read-only request, and returns filtered structured results. Levels 1 and 2 are detailed in Data Handling.

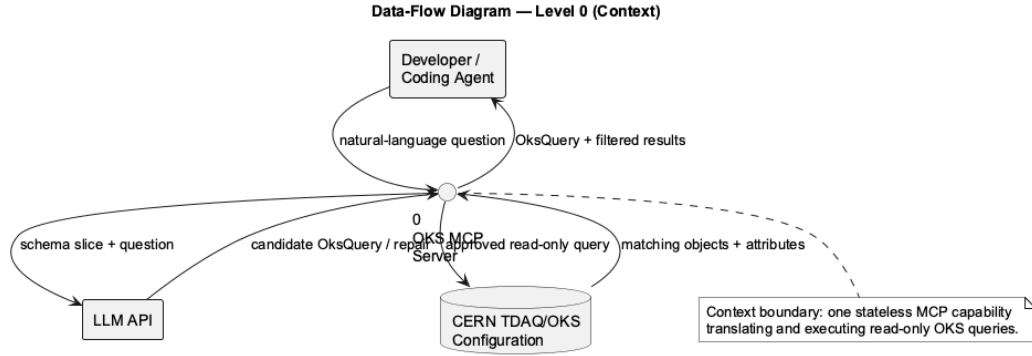


Figure 3.3: Level-0 context data-flow diagram of the OKS MCP Server.

3.3.3 Class Diagram

The class view shows the MCP adapter, service facade, pipeline, and supporting translation, retrieval, validation, and execution components. `OksContext` carries version and schema metadata, while `ExecutionResult` records filtered objects, status, and version. The separation supports independent testing of request handling, model generation, schema checks, and runtime access.

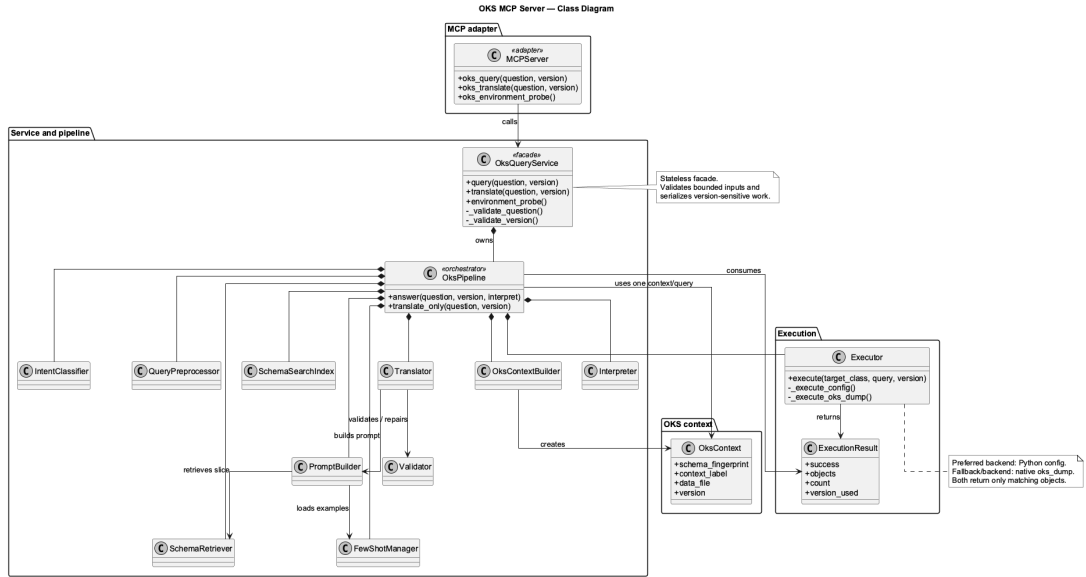


Figure 3.4: Principal classes and collaborations in the merged implementation.

3.3.4 Sequence Diagram

The sequence view shows the deployed path from the user and coding agent, through the local SSH port forward, to the MCP service and pipeline on CERN. The pipeline obtains schema context, asks the LLM for a candidate, validates or repairs it, and executes only the approved read-only query against TDAQ/OKS. Filtered structured results return through the tunnel for agent interpretation.

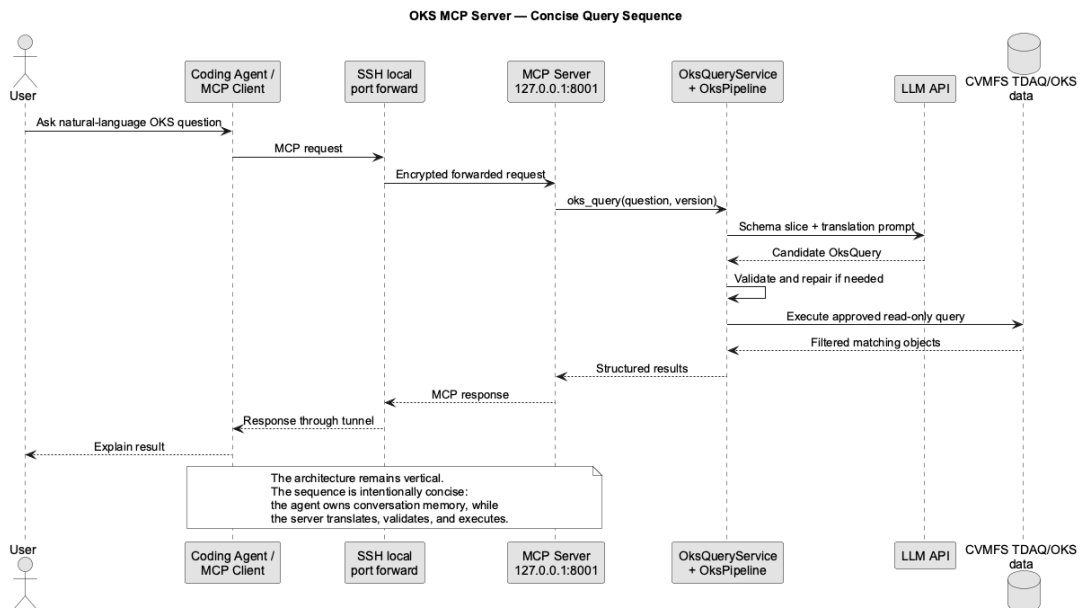


Figure 3.5: Concise sequence of operations for an MCP query request.

3.4 Technology Stack (languages, frameworks, tools, etc)

The system uses the following technologies to support OKS integration, structured translation, and MCP service delivery:

- I. **Python.** Python 3.10 or later is used for the pipeline, service layer, schema processing, validation, and testing.
- II. **OKS/TDAQ integration.** The TDAQ Python config binding provides the primary execution interface, with oks_dump as a fallback and diagnostic tool.
- III. **LLM and structured validation.** The openai package connects to an OpenAI-compatible endpoint. JSON, Pydantic, and the project oks_ast modules support normalisation, semantic validation, repair, and deterministic compilation.
- IV. **Schema retrieval.** Versioned OKS schemas and Git/runtime metadata provide context. Lexical retrieval and rank_bm25 identify relevant classes, attributes, and relationships.
- V. **MCP and deployment.** The Python MCP SDK with FastMCP exposes the query, translation, and environment-probe tools through stdio or Streamable HTTP. Environment variables, python-dotenv, SSH, and tmux support configuration and deployment.
- VI. **Testing.** pytest is used for unit, integration, service, pipeline, and MCP contract tests.

3.5 Development Methodology (agile/waterfall/iterative or suitable one)

The project used iterative prototyping with incremental refinement rather than a strict Agile or Waterfall process. Requirements were clarified through implementation and testing, and the increments were not separately deployed. A prototype was tested against native OKS tools; identified limitations then guided version-aware retrieval, validation and repair, execution, and MCP integration.

Prototype → Test → Refine → Integrate MCP → Validate

3.6 Data Handling (input, storage, processing)

The system handles natural-language questions and approved, version-scoped OKS data without introducing an application database.

3.6.1 Input and Storage

The input is a complete question with an optional current or historical selector. The service validates the question, selector, input length, and result limit; the operator, not the MCP client, chooses the approved data file. TDAQ/OKS schemas and data files remain authoritative. A request-specific schema index, examples, context metadata, and fingerprint are held in memory, while conversation history and configuration objects are not persisted.

3.6.2 Processing and Context Management

The question is classified and used to retrieve a compact, fingerprint-scoped schema slice through exact and lexical matching. The LLM receives only the question and relevant context, not the complete data file. Its JSON query IR is normalised, validated, and repaired within a bounded limit; the deterministic compiler then produces OksQuery. The executor runs the approved query through the Python `config` binding or `oks_dump` and returns bounded, read-only structured results.

3.6.3 Level-1 Data Flow

Figure 3.6 shows the main request flow: validate and classify the question, retrieve schema and examples, translate and validate the query, execute the approved expression against the selected data file, and serialise a bounded MCP response.

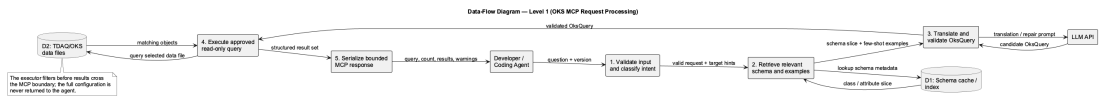


Figure 3.6: Level-1 data-flow diagram for OKS MCP request processing.

3.6.4 Level-2 Translation and Validation Flow

Figure 3.7 expands translation and validation. Target terms select a fingerprinted schema and prompt context; the LLM generates a candidate IR, which is checked for valid classes, attributes, operators, scope, and logic. Only the valid branch reaches compilation and execution; the invalid branch returns diagnostics to a bounded repair prompt.

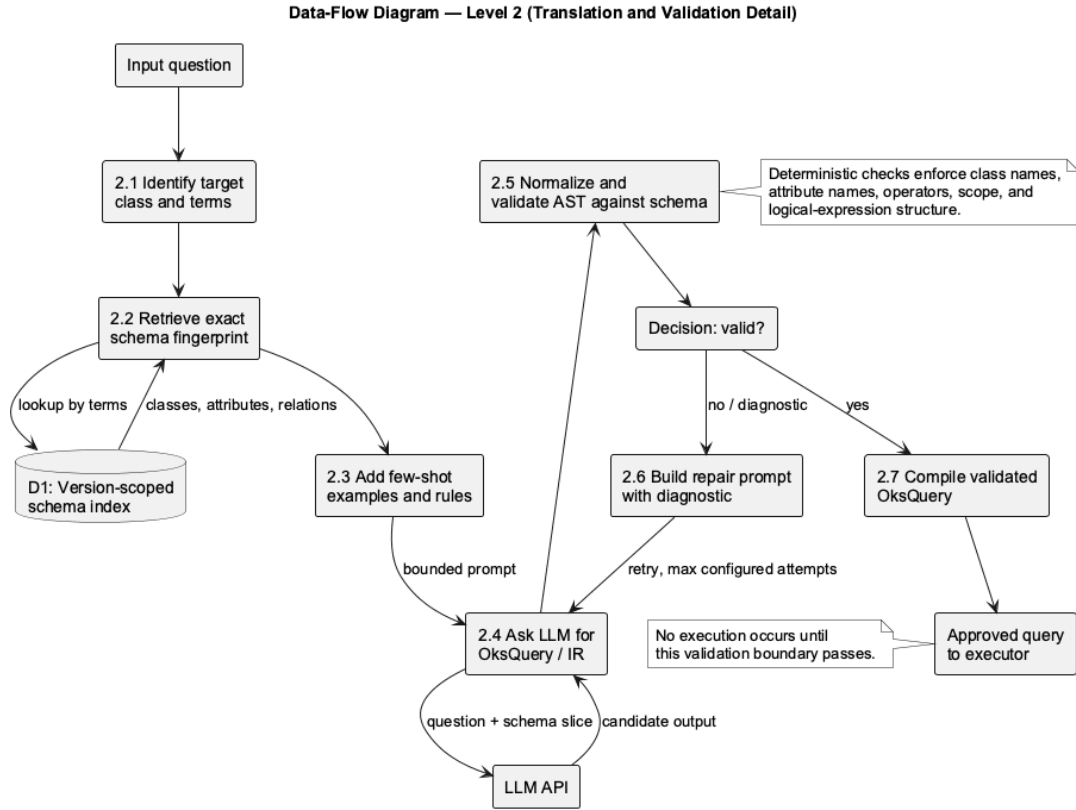


Figure 3.7: Level-2 data-flow diagram for translation and validation.

3.7 Testing and Validation (unit testing, integration testing, user testing)

Testing covered components, the complete pipeline, the service boundary, and the run-time to prevent invalid queries from reaching OKS.

- I. **Unit testing.** Intent, retrieval, version resolution, validation, compilation, and execution components are tested independently.
- II. **Integration testing.** End-to-end tests cover context construction, translation, validation, execution, interpretation, and historical-version consistency.
- III. **Service and MCP testing.** Input limits, version checks, result bounds, serialisation, errors, statelessness, and the three MCP tool contracts are verified.
- IV. **Runtime and evaluation.** Queries are checked against native OKS tools and representative project cases. The MCP-agent workflow is verified for correctness, reproducibility, and safe read-only operation.

The pytest suite contains component, pipeline, service, architecture-invariant, and

MCP-server tests.

3.8 Deployment Setup (hosting, server, environment)

The MCP server runs on CERN lxplus with the ATLAS TDAQ release and OKS files available through CVMFS. The workstation connects through an authenticated SSH tunnel to the loopback-bound MCP endpoint.

- I. **Access and source.** Connect to lxplus through SSH, clone the final GitHub repository, and select the checkout containing the merged MCP implementation.
- II. **Environment preparation.** Source one TDAQ release from CVMFS, create a Python 3.10-or-newer virtual environment, install requirements, and configure the LLM endpoint outside version control. Select the approved OKS_DATA_FILE and repository root.
- III. **Server launch.** The launcher sources TDAQ and `deploy/start_mcp_tmux.sh` starts Streamable HTTP in detached tmux at `127.0.0.1:8001/mcp`; it is not publicly reachable.
- IV. **SSH forwarding.** From the workstation, run `ssh -N -L 18001:127.0.0.1:8001`. The agent uses `http://127.0.0.1:18001/mcp`, while the CERN service remains private.
- V. **Agent connection.** Register the forwarded endpoint in Claude Code or another MCP-compatible agent. It discovers `oks_query`, `oks_translate`, and `oks_environment_probe`; conversation memory remains with the agent.
- VI. **Operations and security.** Check the listener and environment probe before use. Keep the service read-only and loopback-bound, store keys outside Git, and use an approved supervisor for persistent deployment.

CHAPTER 4

IMPLEMENTATION DETAILS

4.1 Requirements Specification(functional/non-functional requirements, constraints)

[Content to be added...]

4.2 System Overview (Description of implemented system, mapping methodology to implementation)

[Content to be added...]

4.3 Development Environment(programming languages, tools, libraries and frameworks used, hardware and software specifications)

[Content to be added...]

4.4 System Implementation(data/input handling, pre-processing steps, algorithm implementation, modules design and integration etc)

[Content to be added...]

4.5 Interface and Visualization

[Content to be added...]

4.6 Deployment and Execution(system setup, execution steps, local/server/cloud setup)

[Content to be added...]

4.7 Challenges and Solutions(technical challenges faced, solutions found)

[Content to be added...]

CHAPTER 5

RESULTS AND DISCUSSION

5.1 Results

[Content to be added...]

5.2 Discussion

[Content to be added...]

5.3 Comparative analysis(if applicable)

[Content to be added...]

CHAPTER 6

CONCLUSIONS AND RECOMMENDATIONS

6.1 Conclusions

[Content to be added...]

6.2 Recommendations/Future Enhancements

[Content to be added...]

REFERENCES

- [1] ATLAS Collaboration, “The git based ATLAS data acquisition configuration service in LHC Run 3,” *Journal of Physics: Conference Series*.
- [2] E. Alexandrov *et al.*, “The ATLAS DAQ system online configurations database service challenge,” *Journal of Physics: Conference Series*, vol. 119, p. 022004, 2007.
- [3] “The OKS persistent in-memory object manager,” ATLAS TDAQ Technical Paper.
- [4] X. Zhu, Y. Xie, Y. Liu, Y. Li, and W. Hu, “Knowledge Graph-Guided Retrieval Augmented Generation,” arXiv:2502.06864, NAACL, 2025.

APPENDICES

[Content to be added...]